Noções de Linguagens de Programação

Algoritmos e Estruturas de Dados

Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Modelos, notações e representações

Algoritmos podem ser representados de várias formas:

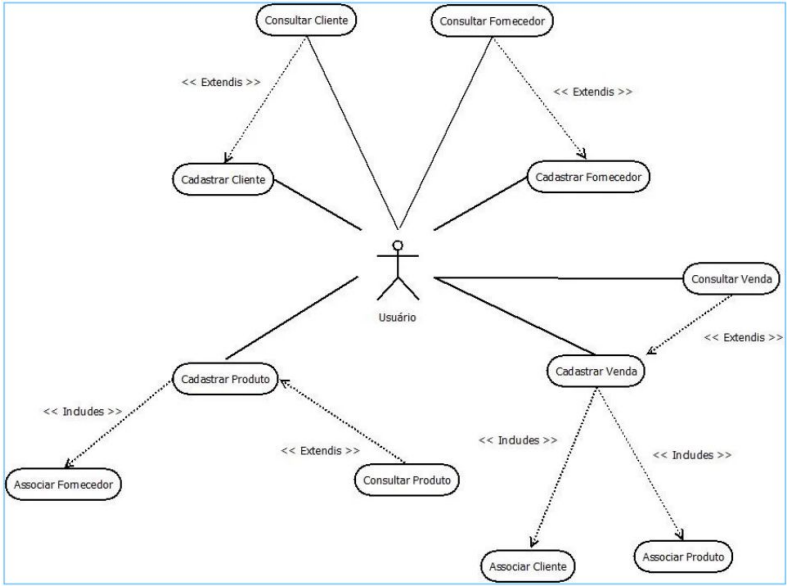
-> pseudocódigo
-> diagramas de fluxo
-> linguagens de modelagem como UML (Unified Modeling Language)
-> descrições verbais detalhadas (linguagem natural)

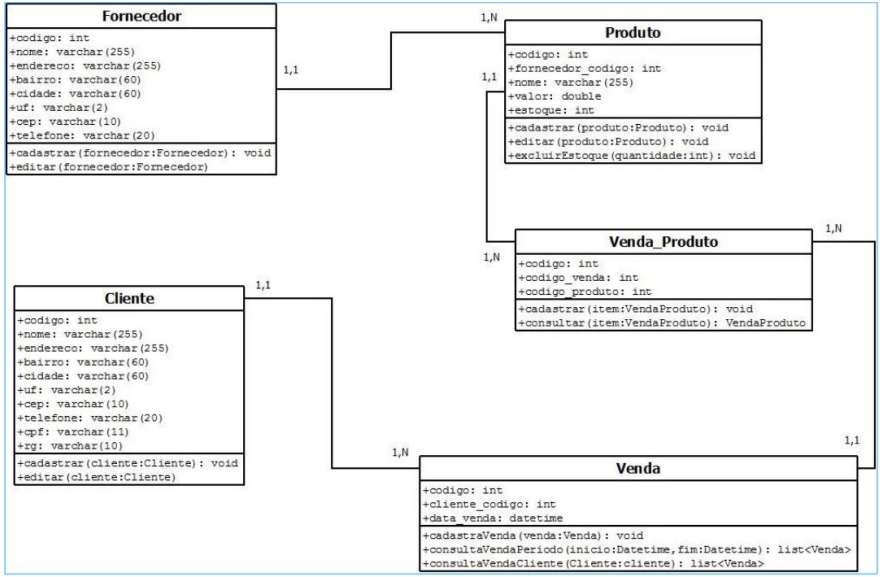
Na disciplina usaremos principalmente a linguagem Python

Modelos, notações e representações

Exemplos da UML (Unified Modeling Language)

-> Caso de uso e Diagrama de Classes:





carlos.melo@upe.br

Modelos, notações e representações

Notações são sistemas de símbolos e regras que fornecem uma maneira padronizada de representar algoritmos..

Elas são usadas para expressar a estrutura e o funcionamento dos algoritmos de uma forma mais concisa e compreensível.

Pseudocódigo é um exemplo comum de notação.

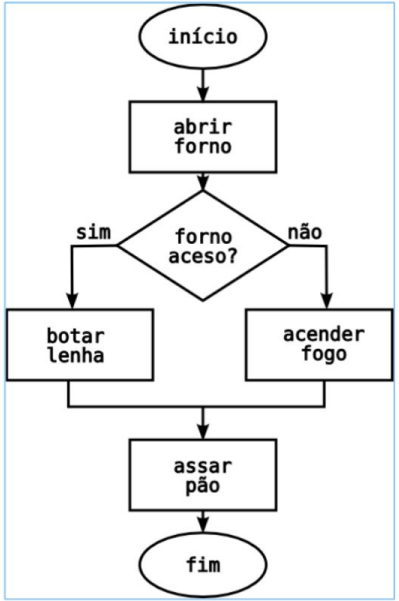
```
para i de 1 até n
    imprima i
```

```
for i in range(n):
    print(i)
```

Modelos, notações e representações

Representações referem-se às diferentes formas visuais ou textuais usadas para apresentar algoritmos.

Diagramas de fluxo são uma representação visual comum, onde diferentes etapas do algoritmo e setas indicam o fluxo de controle.



Modelos, notações e representações

Trechos de código em uma linguagem específica também são representações de algoritmos

Em C:

```
printf("Hello, World!\n");
```

Em Java:

```
System.out.println("Hello, World!");
```

Em Python:

```
print("Hello, World!")
```

Em Portugol:

```
escreva("Hello, World!")
```

Noções de linguagens de programação

Linguagens de programação são usadas para instruir o computador a executar tarefas.

o mundo conforme o conhecemos depende de linguagens de programação, pois todo o software executando em todos os computadores e dispositivos foi escrito em alguma linguagem de programação.

algoritmo -> código -> execução

Noções de linguagens de programação

Antes de executar, um programa precisa ser traduzido para linguagem de máquina.

Os sistemas de software que fazem essa tradução são os **compiladores e os interpretadores.**

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

-> COMPILADOR ->

```
01001000    01100101    01101100
01101100    01101111    00101100
00100000    01010111    01101111
01110010    01101100    01100100
00100001
```

Noções de linguagens de programação

Existem duas formas principais de executar programas:

Compilador:
-> traduz o programa inteiro antes da execução
-> gera um executável

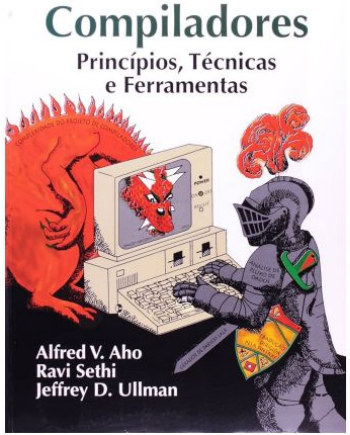
Interpretador:
-> executa o programa linha a linha
-> não gera executável

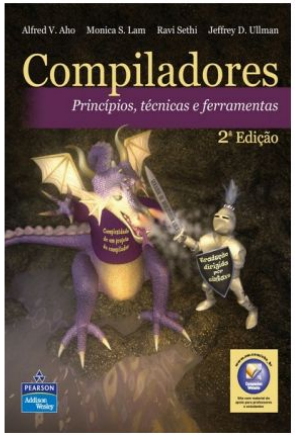
Exemplos linguagens **compiladas** : C, C++, C#, Java...
Exemplos de linguagens **interpretadas** : Python, JavaScript

Noções de linguagens de programação

Um compilador é um programa que recebe como entrada um programa escrito em uma linguagem de programação (linguagem **fonte**) e o traduz para um programa equivalente em outra linguagem (linguagem **objeto**).

Um papel importante do compilador é relatar quaisquer erros no programa fonte detectados durante esse processo de <u>tradução</u>.



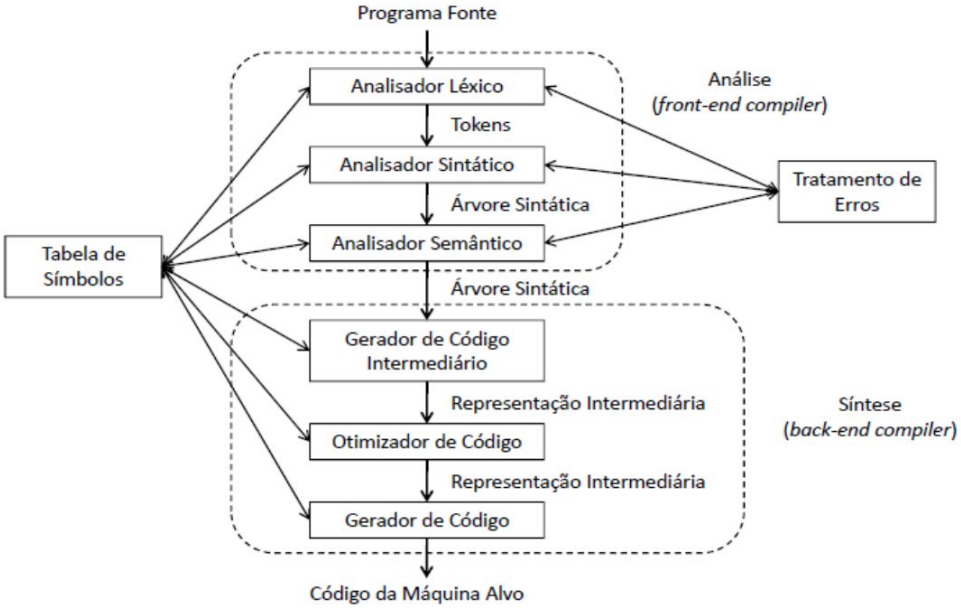


Noções de linguagens de programação

Compilador (C, C++)	Interpretador (Python, JS)
- Traduz o programa inteiro de uma vez. - Cria um arquivo executável (.exe) - Rápida execução - Debugging: Requer re-compilação	- Traduz linha a linha durante execução - Nenhum arquivo executável é criado - Execução <u>mais lenta</u> - Debugging: Resposta imediata e fácil diagnóstico

Noções de linguagens de programação

Estrutura básica de um compilador:



Fonte: adaptado de Price e Toscani (2001).

carlos.melo@upe.br

Noções de linguagens de programação

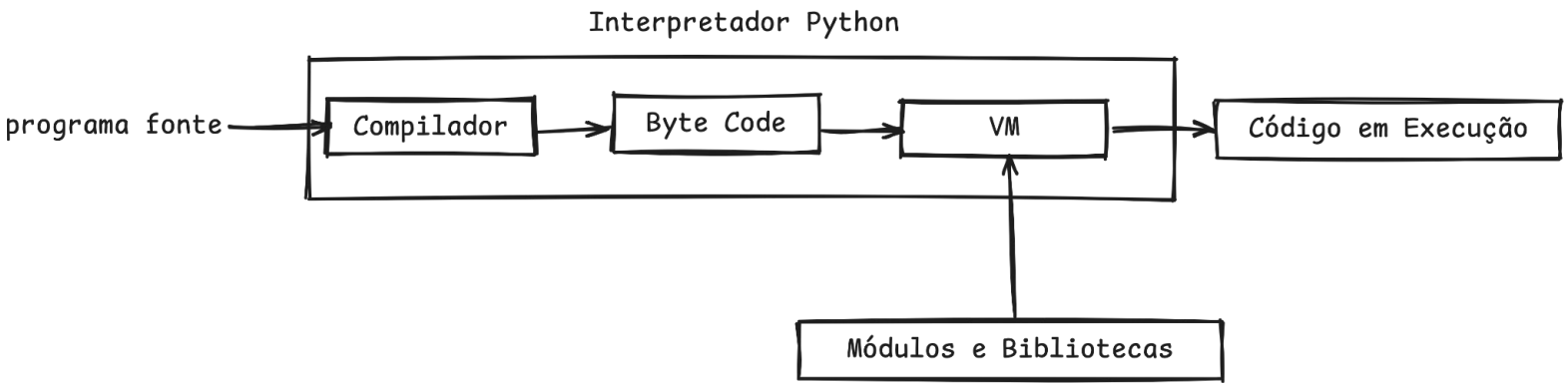
A fase de **análise** de um compilador subdivide um programa fonte em partes e produz uma representação interna chamada código intermediário.

A fase de **síntese** traduz o código intermediário para o programa objeto.

A análise é organizada em torno da "sintaxe" da linguagem a ser compilada. A **sintaxe** de uma linguagem de programação descreve a forma apropriada dos seus programas, enquanto a **semântica** da linguagem define o que seus programas significam; ou seja, o que cada programa faz quando é executado.

Noções de linguagens de programação

Estrutura do interpretador Python:



Python

- **Python** é uma linguagem de propósito geral criada em 1989 por Guido van Rossum.
- Dentre as principais aplicações estão ciência e análise de dados, além de desenvolvimento web e scripts de automação.
- O nome da linguagem é uma homenagem ao programa de televisão Monty Python's Flying Circus.

Python

As duas principais características da linguagem são:

1. Python é case sensitive (Eu != eU)
2. Espaçamento é importante (4 espaços vs. 1 tab)

Operadores aritméticos

- + (adição)
- - (subtração)
- * (multiplicação)
- / (divisão)
- % (módulo/resto da divisão)
- ** (Exponenciação)
- // (Divisão com arredondamento para baixo)

A ordem de prioridade de operações é a mesma da matemática: **PEMDAS**

- **P**arênteses, **E**xponenciais, **M**ultiplicação/**D**ivisão, **A**dição/**S**ubtração

Variáveis e Atribuição

Uma variável é um rótulo para algum valor armazenado em memória

- `idade = 28`
  - A variável **idade** recebe o valor inteiro **28**.
- Também é possível realizar atribuições múltiplas, **por mais que eu não te permita fazer isso nas atividades da disciplina algo assim funcionaria** :
  - `a, b, c = 1, 2, 3`

Dicas:

- Use sempre nomes sugestivos para suas variáveis
- Apenas letras, números e *underlines* são permitidos em nomes de variáveis
- Nomes de variáveis não podem iniciar por números
- Você não pode usar palavras reservadas da linguagem como nome para variáveis

Variáveis e Atribuição

Padrão ***snake_case*** (palavras em minúsculas separadas por *underline*):

- **`minha_idade`** = 18
- **`distancia_em_km`** = 120

camelCase
snake_case
PascalCase

Variáveis e Atribuição

Atribuição (=, +=, -= e *=)

- x = 2
- x += 2 equivalente à x = x + 2
- x -= 2 equivalente à x = x - 2
- x *= 2 equivalente à x = x * 2

Tipos de Dados

Há dois tipos padrão para lidar com valores numéricos: **int** e **float**

- **int** é para valores inteiros (2, 3, 182738, ...)
- **float** para valores de ponto flutuante (1.0, 5656.12, 0.35, ...)

É possível realizar casting entre tipos o que pode gerar perda de valores:

- `x = int(1.22) # x = 1`
- `y = float(1) # x = 1.0`

Também é possível verificar o tipo de uma variável através da função `type(var)`

Exercício

Qual é o tipo de ***z***?

- **`float`**

```
x = 10
y = 3.5
z = x + y
print(type(z))
```

Boas práticas

O guia de boas práticas da linguagem é o Python Enhancement Proposals 8 (PEP8):

Python Enhancement Proposals | Python » PEP Index » PEP 8

Search

Contents

- Introduction
- A Foolish Consistency is the Hobgoblin of Little Minds
- Code Lay-out
  - Indentation
  - Tabs or Spaces?
  - Maximum Line Length
  - Should a Line Break Before or After a Binary Operator?
  - Blank Lines
  - Source File Encoding
  - Imports
  - Module Level Dunder Names
- String Quotes
- Whitespace in Expressions and Statements
  - Pet Peeves
  - Other Recommendations
- When to Use Trailing Commas
- Comments
  - Block Comments
  - Inline Comments
  - Documentation Strings
- Naming Conventions
  - Overriding Principle
  - Descriptive: Naming Styles
  - Prescriptive: Naming Conventions

PEP 8 – Style Guide for Python Code

Author: Guido van Rossum <guido at python.org>, Barry Warsaw <barry at python.org>, Alyssa Coghlan <ncoghlan at gmail.com>

Status: Active
Type: Process
Created: 05-Jul-2001
Post-History: 05-Jul-2001, 01-Aug-2013

▸ Table of Contents

Introduction

This document gives coding conventions for the Python code comprising the standard library in the main Python distribution. Please see the companion informational PEP describing style guidelines for the C code in the C implementation of Python.

This document and PEP 257 (Docstring Conventions) were adapted from Guido's original Python Style Guide essay, with some additions from Barry's style guide [2].

This style guide evolves over time as additional conventions are identified and past conventions are rendered obsolete by changes in the language itself.

Many projects have their own coding style guidelines. In the event of any conflicts, such project-specific guides take precedence for that project.

Booleanos e Operadores Lógicos e de Comparação

O tipo booleano (*bool*) armazena valores que podem ser **True** ou **False**

Existem seis operadores de comparação no Python que retorna um booleano

- Menor que: **4 < 2** retorna **False**
- Maior que: **4 > 2** retorna **True**
- Menor ou igual a: **2 <= 2** retorna **True**
- Maior ou igual a: **2 >= 4** retorna **False**
- Igual a: **4 == 2** retorna **False**
- Diferente de: **4 != 2** retorna **True**

```
1 idade = 18
2 print(idade >= 18)
```

Booleanos e Operadores Lógicos e de Comparação

Existem outros três operadores lógicos no Python que retorna um booleano

- E (and): **4 < 2 and 5 == 5** -> **False # Avalia se todas as declarações retornam True**
- OU (or): **4 < 2 or 4 == 4** -> **True # Avalia se pelo menos uma declaração é True**
- NÃO (not): **not 4 < 2** -> **True # Inverte um valor booleano**

```
1 idade = 20
2 tem_carteira = True
3
4 print(idade >= 18 and tem_carteira)
```

Exercício

Qual é a saída do código abaixo?

- **False**

```
1 x = 5
2 y = 10
3
4 print(x < y and y > 20)
```

Tipos de Dados

Para lidar com cadeias de caracteres em Python temos: **str**

- Uma *string* pode ser declarada entre aspas ‘simples’ e aspas “duplas”.

- A recomendação é o uso de aspas duplas porque podemos nos deparar com situações como: **“pai d’égua”**.

É possível realizar casting entre tipos o que pode gerar perda de valores:

- **x = str(1.22) # x = ‘1.22’**
- **y = “meu texto” # x = meu texto**

Também é possível verificar o tamanho de uma *string* através da função **len**(str)

```
1 nome = "Carlos"
2 len(nome)
```

Métodos de Strings

Métodos são como funções para classes e tipos

As *strings* possuem um conjunto de métodos associados

-> os outros tipos também possuem vários métodos...

```
>>> meu_texto = "bom dia"
>>> meu_texto.
meu_texto.capitalize()    meu_texto.isalpha()       meu_texto.ljust(          meu_texto.rsplit(
meu_texto.casefold()      meu_texto.isascii()       meu_texto.lower()         meu_texto.rstrip(
meu_texto.center(         meu_texto.isdecimal()     meu_texto.lstrip(         meu_texto.split(
meu_texto.count(          meu_texto.isdigit()       meu_texto.maketrans(      meu_texto.splitlines(
meu_texto.encode(         meu_texto.isidentifier()  meu_texto.partition(      meu_texto.startswith(
meu_texto.endswith(       meu_texto.islower()       meu_texto.removeprefix(   meu_texto.strip(
meu_texto.expandtabs(     meu_texto.isnumeric()     meu_texto.removesuffix(   meu_texto.swapcase()
meu_texto.find(           meu_texto.isprintable()   meu_texto.replace(        meu_texto.title()
meu_texto.format(         meu_texto.isspace()       meu_texto.rfind(          meu_texto.translate(
meu_texto.format_map(     meu_texto.istitle()       meu_texto.rindex(         meu_texto.upper()
meu_texto.index(          meu_texto.isupper()       meu_texto.rjust(          meu_texto.zfill(
meu_texto.isalnum()       meu_texto.join(           meu_texto.rpartition(
```

```
>>> meu_texto = "bom dia"
>>> meu_texto.islower()
True
>>> meu_texto.count("a")
1
>>> meu_texto.find("a")
6
>>> meu_texto.split()
['bom', 'dia']
```

Exercícios

Q1: Algoritmos podem ser representados de diversas formas para facilitar sua compreensão e implementação. Entre as alternativas abaixo, qual **não** é considerada uma forma comum de representação de algoritmos?

A) Pseudocódigo

B) Diagramas de fluxo

C) Linguagem natural

D) Linguagem binária executável

E) UML

D) Linguagem binária é resultado de compilação, não representação conceitual.

Exercícios

Q2: Um compilador recebe como entrada um programa em linguagem fonte e gera um programa equivalente em linguagem objeto. Sobre as etapas de compilação, assinale a alternativa correta.

A) A análise léxica verifica o significado do programa.

B) A análise sintática verifica se os *tokens* formam estruturas válidas.

C) A análise semântica gera diretamente o código de máquina.

D) A geração de código ocorre antes da análise sintática.

E) A análise léxica gera o programa executável.

B).

Exercícios

Q4: Considere o código Python abaixo:

Qual é o valor final de **x**?

```
>>> x = 10
>>> x += 5
>>> x *= 2
```

A) 20

B) 25

C) 30

D) 40

E) 50

C)
10 + 5 = 15
15 × 2 = 30

Exercícios

Q5: Considere o código Python abaixo:

Qual é o resultado da expressão?

```
>>> 4 < 2 or 5 == 5
```

A) False

B) True

C) None

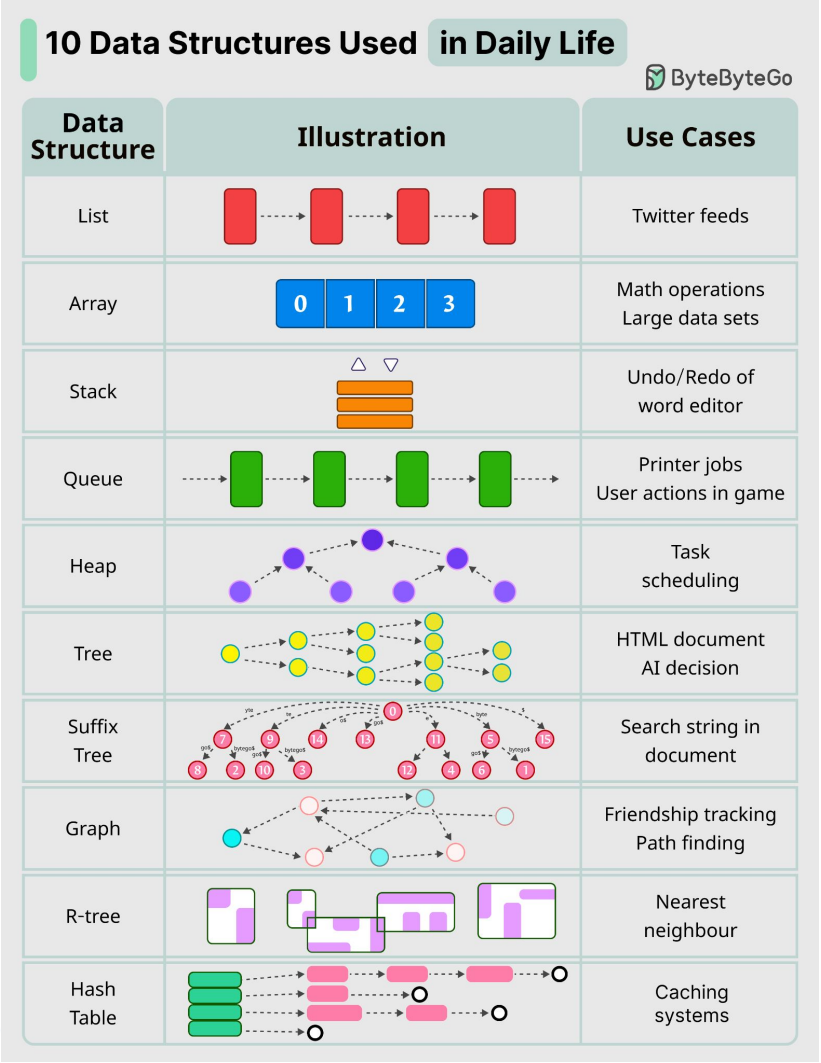
D) 0

E) Erro de Sintaxe

B)

Estruturas de Dados

São coleções de dados organizados (geralmente relacionados) dispostos das mais diferentes formas.



carlos.melo@up

Listas

As Listas são uma coleção de dados ***mutável*** e ***ordenada***, além de uma das estruturas de dados mais comuns em Python

<<Listas são semelhantes a Arrays>>
Porém são mais flexíveis (dados de diferentes tipos)

Listas podem ser criadas através de colchetes `[]` ou do ***construtor*** `list()`

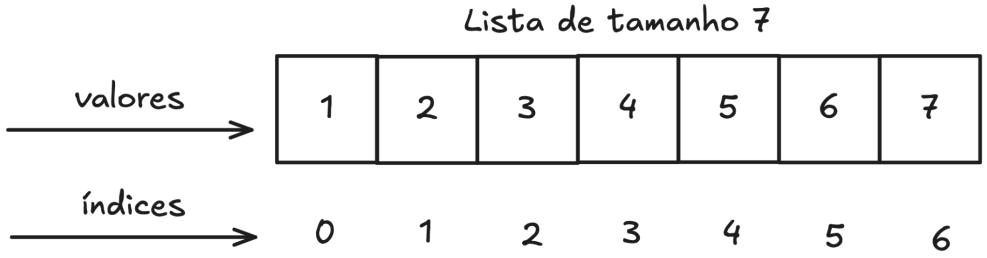
Exemplo de lista:

- **`lista_baguncada`** `= [1, 1.1, “hoje é segunda-feira”, True]`
- **`dias_da_semana`** `= [“segunda-feira”, “terça-feira”, “quarta-feira”]`

Listas

Os elementos de uma lista podem ser acessados através de seu índice (posição).

Os índices de listas iniciam na posição 0 (zero) e vão até a sua quantidade de itens - 1.



Listas

O acesso ao valor correspondente é direto e é feito através da passagem do respectivo índice dentro de colchetes

```
>>> dias_da_semana = [“segunda-feira”, “terça-feira”, “quarta-feira”]
>>> dias_da_semana[2]
>>> “quarta-feira”
```

Caso o índice inexista um `IndexError` é retornado.

```
>>> dias_da_semana[3]
>>> IndexError: list index out of range
```

Listas

Também é possível realizar um moonwalker na lista e passar índices negativos

```
>>> dias_da_semana = [“segunda-feira”, “terça-feira”, “quarta-feira”]
>>> dias_da_semana[-1]
>>> “quarta-feira”
>>> dias_da_semana[-2]
>>> “terça-feira”
```

Listas

Para verificar se um elemento está ou não na lista usamos os operadores **in** e **not in**

```
>>> dias_da_semana = [“segunda-feira”, “terça-feira”, “quarta-feira”]
>>> “segunda-feira” in dias_da_semana
>>> True
>>> “quinta-feira” in dias_da_semana
>>> False
>>> “sexta-feira” not in dias_da_semana
>>> True
```

Esses operadores literalmente executam a busca simples (linear)

Mutabilidade das Listas

Mutabilidade se refere ao fato de podermos ou não alterar um *objeto* uma vez que ele é criado. Se podemos alterar o objeto então é **mutável**, se não podemos é **imutável**.

Listas são mutáveis.

```
>>> dias_da_semana = [“segunda-feira”, “terça-feira”, “quarta-feira”]
>>> dias_da_semana[0] = “sexta-feira”
>>> print(dias_da_semana)
>>> [“sexta-feira”, “terça-feira”, “quarta-feira”]
```

Ordem das Listas

Ordem é sobre o valor dos dados em um objeto serem acessados em uma ordem, os índices representam uma ordem.

listas são ordenadas.

```
>>> dias_da_semana = ["segunda-feira", "terça-feira", "quarta-feira"]
>>> dias_da_semana[0]
>>> "segunda-feira"
>>> dias_da_semana[1]
>>> "terça-feira"
```

-> Podemos inserir um novo elemento na lista pelo método **append()**, que insere no final, e também através do **insert()** que insere em um índice específico.
-> A remoção pode ser feita através do **pop()** que remove o último elemento da lista ou o índice passado e **remove()** que retira a primeira ocorrência de uma valor.

Funções comuns em Listas

As principais funções usadas em listas são:

1. **`len()`** -> retorna o número de elementos na lista
2. **`max()`** -> retorna o maior elemento na lista
3. **`min()`** -> retorna o menor elemento na lista
4. **`sorted()`** -> retorna uma cópia ordenada da lista

Exercícios

```python
# qual a saída?
numeros = [10, 20, 30, 40]

print(numeros[2])
```

```python
# qual a saída?
numeros = [1, 2, 3]
numeros.append(4)

print(numeros)
```

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS